



Security Assessment Report

Orca Whirlpools

PR 1366

August 18, 2026

Summary

The Sec3 team was engaged to conduct an incremental review of PR 1366 of the Orca Whirlpools.

The artifact of the audit was the source code of the following programs, excluding tests, in <https://github.com/orca-so/whirlpools>.

The initial audit focused on the following versions and revealed 1 issues or questions.

Task	Type	Commit
PR1366 - Prepare for rent reduction starting with Agave 4.2	Solana	e5ee72f → d12f6ac

This report provides a detailed description of the findings and their respective resolutions.

Table of Contents

Result Overview 3

Findings in Detail 4

 [I-01] Rent fallback may temporarily DoS wSOL reward collection 4

Appendix: Methodology and Scope of Work 7

Result Overview

Issue	Impact	Status
PR1366 - PREPARE FOR RENT REDUCTION STARTING WITH AGAVE 4.2		
[I-01] Rent fallback may temporarily DoS wSOL reward collection	Info	Acknowledged

Findings in Detail

PR1366 - PREPARE FOR RENT REDUCTION STARTING WITH AGAVE 4.2

[I-01] Rent fallback may temporarily DoS wSOL reward collection

Identified in commit [351a797](#).

In PR1366, the Whirlpool program updated the rent amount used to create token accounts to a fixed value of `2_039_280` lamports. This change accounts for both the SIMD-0437 rent reduction and a potential fallback. By preserving the pre-reduction rent amount, token accounts should remain rent-exempt even if rent decreases and is later restored.

```
/* programs/whirlpool/src/util/rent.rs */
058 | pub fn get_native_mint_token_account_minimum_rent_amount() -> Result<u64> {
059 |     #[cfg(target_os = "solana")]
060 |     assert_expected_rent_config()?;
061 |     Ok(2_039_280)
062 | }

/* programs/whirlpool/src/util/token_2022.rs */
463 | pub fn initialize_vault_token_account<'info>(<
465 |     vault_token_account: &Signer<'info>,
470 | ) -> Result<()> {
494 |     let lamports = if spl_token::native_mint::check_id(&vault_mint_account_info.key()) {
495 |         // Enforce the pre-Rent Reduction lamport level to prevent the lamports available as the WSOL balance
↳ from decreasing
496 |         // if rent increases due to the Rent Reduction Fallback.
497 |         get_native_mint_token_account_minimum_rent_amount()?
498 |     } else {
499 |         Rent::get()?.minimum_balance(space as usize)
500 |     };
502 |     // create account
503 |     safe_create_account(
506 |         vault_token_account.to_account_info(),
508 |         lamports,
511 |     )?;
527 |     // initialize token account
528 |     invoke(
529 |         &spl_token_2022::instruction::initialize_account3(
531 |             vault_token_account.key,
534 |         )?,
541 |     );
```

When SIMD-0437 is activated, for wSOL token accounts, the surplus lamports can be reflected in the token account balance, as both `sync_native` and `initialize_account3` calculate the rent-exempt reserve using the default reduced value returned by `minimum_balance`:

```

/* https://github.com/solana-program/token/blob/p-token@v1.0.0/program/src/processor.rs */
757 | pub fn process_sync_native(program_id: &Pubkey, accounts: &[AccountInfo]) -> ProgramResult {
762 |     let rent_exempt_reserve = if let Ok(rent_sysvar_info) = next_account_info(account_info_iter)
763 |     {
764 |         let rent = Rent::from_account_info(rent_sysvar_info)?;
765 |         rent.minimum_balance(native_account_info.data_len())
766 |     } else {
767 |         Rent::get()?.minimum_balance(native_account_info.data_len())
768 |     };
772 |     if native_account.is_native.is_some() {
773 |         let new_amount = native_account_info
774 |             .lamports()
775 |             .checked_sub(rent_exempt_reserve)
776 |             .ok_or(TokenError::Overflow)?;
778 |         native_account.amount = new_amount;
779 |     } else {
780 |         return Err(TokenError::NonNativeNotSupported.into());
781 |     }

/* https://github.com/solana-program/token/blob/p-token@v1.0.0/program/src/processor.rs */
083 | fn _process_initialize_account(
088 | ) -> ProgramResult {
126 |     if is_native_mint {
127 |         let rent_exempt_reserve = rent.minimum_balance(new_account_info.data_len());
129 |         account.amount = new_account_info
130 |             .lamports()
131 |             .checked_sub(rent_exempt_reserve)
132 |             .ok_or(TokenError::Overflow)?;
133 |     } else {
136 |     };

```

This does not create an issue for Whirlpool swap or liquidity-management operations because these flows do not depend on the vault's exact token balance (i.e., the `amount` field). Even if the surplus SOL reserved by the fixed rent amount is converted into WSOL through `sync_native`, the resulting WSOL remains in the vault.

However, this behavior may create a risk for WSOL reward vaults. In `collect_reward`, the amount transferred to the user is determined using `reward_vault.amount` and the vault's token balance may be transferred out.

```

/* programs/whirlpool/src/instructions/v2/collect_reward.rs */
058 | pub fn handler<'info>(<
059 |     ctx: Context<'_, '_, '_, 'info, CollectRewardV2<'info>>,
060 |     reward_index: u8,
061 |     remaining_accounts_info: Option<RemainingAccountsInfo>,
062 | ) -> Result<()> {
078 |     let (transfer_amount, updated_amount_owed) = calculate_collect_reward(
079 |         position.reward_infos[index],
080 |         ctx.accounts.reward_vault.amount,
081 |     );

/* programs/whirlpool/src/instructions/v2/collect_reward.rs */
099 | fn calculate_collect_reward(position_reward: PositionRewardInfo, vault_amount: u64) -> (u64, u64) {
100 |     let amount_owed = position_reward.amount_owed;

```

```

101 |     let (transfer_amount, updated_amount_owed) = if amount_owed > vault_amount {
102 |         (vault_amount, amount_owed - vault_amount)
103 |     } else {
104 |         (amount_owed, 0)
105 |     };
107 |     (transfer_amount, updated_amount_owed)
108 | }

```

Consider the following scenarios:

Scenario 1

1. The `Whirlpool` owner creates a `wSOL` reward token account using the fixed rent amount.
2. `SIMD-0437` is activated, and a lower rent-exempt reserve is required for the `wSOL` reward token account.
3. Someone calls `sync_native` on the `wSOL` reward token account, causing the extra rent lamports to be reflected as `wSOL` token amount.
4. During `collect_reward`, this amount may be transferred out as reward tokens.
5. The rent reduction is later reverted; the reward vault may no longer contain enough lamports to remain rent-exempt.

Scenario 2

1. `SIMD-0437` is activated.
2. The `Whirlpool` owner creates a `wSOL` reward vault using the fixed rent amount.
3. During initialization, the surplus rent reserve is reflected in the vault's `wSOL` token balance.
4. A subsequent `collect_reward` call may transfer this surplus out as reward tokens.
5. The rent reduction is later reverted; the reward vault may no longer contain enough lamports to remain rent-exempt.

In both scenarios, reward collection therefore fails until the missing lamports are restored.

Resolution

The team acknowledged this finding as a valid but unlikely edge case. Exploitation would require both a `SIMD-0438` rent fallback and rarely used `WSOL` reward vaults to over-emit. Because the impact is limited and recovery is straightforward, the team plans to document a recovery procedure: replenish the vault with a small amount of `SOL` and call `sync_native` to recalculate its balance using the restored rent requirement.

Appendix: Methodology and Scope of Work

Assisted by the Sec3 Scanner developed in-house, the manual audit particularly focused on the following work items:

- Check common security issues.
- Check program logic implementation against available design specifications.
- Check poor coding practices and unsafe behavior.
- The soundness of the economics design and algorithm is out of scope of this work

DISCLAIMER

The instance report ("Report") was prepared pursuant to an agreement between Coderect Inc. d/b/a Sec3 (the "Company") and Orca Management Company, S.A (the "Client"). This Report solely includes the results of a technical assessment of a specific build and/or version of the Client's code specified in the Report ("Assessed Code") by the Company. The sole purpose of the Report is to provide the Client with the results of the technical assessment of the Assessed Code. The Report does not apply to any other version and/or build of the Assessed Code. Regardless of the contents of the Report, the Report does not (and should not be interpreted to) provide any warranty, representation or covenant that the Assessed Code: (i) is error and/or bug free, (ii) has no security vulnerabilities, and/or (iii) does not infringe any third-party rights. Moreover, the Report is not, and should not be considered, an endorsement by the Company of the Assessed Code and/or of the Client. Finally, the Report should not be considered investment advice or a recommendation to invest in the Assessed Code and/or the Client.

This Report is considered null and void if the Report (or any portion thereof) is altered in any manner.

ABOUT

The Sec3 audit team comprises a group of computer science professors, researchers, and industry veterans with extensive experience in smart contract security, program analysis, testing, and formal verification. We are also building automated security tools that incorporate static analysis, penetration testing, and formal verification.

At Sec3, we identify and eliminate security vulnerabilities through the most rigorous process and aided by the most advanced analysis tools.

For more information, check out our [website](#) and follow us on [twitter](#).

